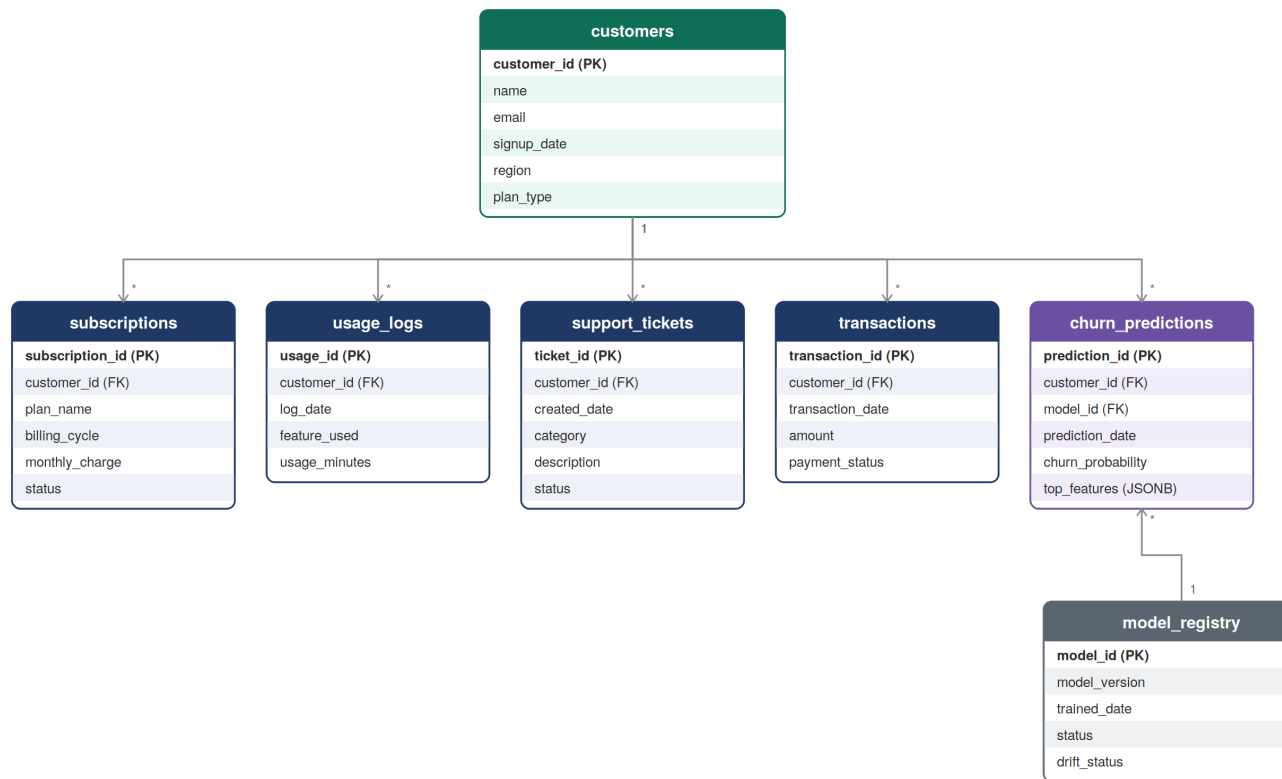


Kairos Entity Relationship Diagram

Phase 2 deliverable: the governed data model behind Kairos, explained table by table.



customers (core entity)

This is the anchor table that everything else connects back to. Every subscription, usage record, support ticket, transaction, and churn prediction belongs to exactly one customer, identified by `customer_id`. Losing or duplicating a `customer_id` here would break every other table, which is why this table is the center of the whole schema.

subscriptions

Stores what plan a customer is on, how they are billed, and whether the subscription is currently active or cancelled. One customer can have multiple subscription records over time, for example if they upgrade, downgrade, or resubscribe later.

usage_logs

Tracks how actively a customer is actually using the product: which features they touch, how many minutes they spend, and when. This is one of the richest sources of behavioral signal for the churn model, since drop-off in usage is often an early warning sign before a customer formally cancels.

support_tickets

Stores every support interaction a customer has had, including a free-text description field. This description field is what gets embedded and searched semantically later by the chatbot, so it can answer fuzzy questions like customers who complained about billing.

transactions

Stores every payment event tied to a customer: the amount, the date, and whether the payment succeeded or failed. Failed or irregular payments are often a strong churn signal on their own.

churn_predictions

This table is the output of the machine learning model, not raw business data. For each prediction it stores which customer it is about, which model version produced it, the churn probability, and a top_features field that holds the SHAP explanation behind that specific prediction. This is the exact field the chatbot reads from when someone asks why a customer is at risk.

model_registry

Keeps track of every trained version of the churn model: when it was trained, its current status, and whether drift has been detected. Each row in churn_predictions links back to exactly one model version here, which is what allows the system to know if a given prediction came from a model that is now outdated.

How the tables connect

Every table except model_registry links back to customers in a one-to-many relationship, meaning one customer can have many subscriptions, many usage records, many tickets, many transactions, and many churn predictions over time. Separately, model_registry links to churn_predictions in its own one-to-many relationship, since one trained model version can produce many predictions across many customers, but each individual prediction was produced by exactly one specific model version. This second link is what lets Kairos trace any prediction back to the exact model that made it, and check whether that model is still trustworthy.